Foundations of Artificial Intelligence
Appunti

Giovanni "Qua' Qua' dancer" Palma

Contents

Chapter 1 **Introduzione all'Intelligenza Artificiale** Page

1.1 Che cos'è l'intelligenza?

1.2 Turing: le macchine possono pensare?
Oltre il test di Turing — • La previsione di Turing —

1.3 Classificazione dei sistemi di IA

1.4 Nascita e definizioni dell'IA

1.5 Breve storia dell'IA

1.6 I due approcci principali all'IA

1.7 Ragionamento e logica

1.8 Logica, dimostrazione automatica e Prolog
Tre letture del concetto di algoritmo — • Linguaggi dichiarativi e Prolog — • Struttura di un programma Prolog — • Deduzione e induzione in programmazione logica — • Abduzione in programmazione logica — • Prolog: successi e limiti —

1.9 Machine learning
Le tre grandi famiglie — • Che cosa si può fare con il machine learning? — • Task: classificazione — • Task: regressione — • Task: clustering —

1.10 Apprendimento simbolico e sub-simbolico
Apprendimento simbolico: alberi di decisione (ID3, C4.5) — • Il neurone artificiale — • Il percettrone a due input — • Reti neurali multi-livello —

1.11 Deep Neural Networks e Deep Learning

1.12 Foundation Models

1.13 Agentic AI

1.14 Automated Planning
Esempio: il mondo dei blocchi — • Tecniche di planning —

1.15 Swarm intelligence e metafore naturali
Algoritmi genetici ed evolutivi —

1.16 Reinforcement learning

1.17 Riepilogo

Chapter 1

Introduzione all'Intelligenza Artificiale

Lezione 1 – M. Milano, DISI, Università di Bologna. Questo capitolo copre la panoramica introduttiva del corso: che cosa si intende per intelligenza, il test di Turing, le classificazioni dei sistemi di IA, i due grandi approcci (simbolico e connessionista), le forme di ragionamento, la programmazione logica, il machine learning, il planning automatico e le metafore naturali (swarm intelligence, algoritmi genetici, reinforcement learning).

1.1 Che cos'è l'intelligenza?

Definition 1.1.1: Intelligenza

L'intelligenza è stata definita in molti modi, tra cui: la capacità di **logica**, **comprensione**, **autocoscienza**, **apprendimento**, **conoscenza emotiva**, **ragionamento**, **pianificazione**, **creatività**, **pensiero critico** e **problem solving**.
Più in generale, può essere descritta come l'abilità di *percepire o inferire informazione*, e di *ritenerla come conoscenza* da applicare per ottenere *comportamenti adattivi* all'interno di un ambiente o di un contesto.

Note:

La definizione è volutamente vaga: non esiste un consenso su che cosa sia l'intelligenza, e di conseguenza non esiste una definizione univoca di *Intelligenza Artificiale*. Questa ambiguità è il motivo per cui Turing sposta la domanda dal *che cos'è* al *come lo riconosciamo*.

1.2 Turing: le macchine possono pensare?

Nel 1950 Alan Turing pubblica *Computing Machinery and Intelligence*. Il problema di partenza è duplice: definire una **macchina universale** e definire che cosa significa **pensare** (termine ambiguo). Turing aggira il secondo problema sostituendolo con un criterio operativo.

Definition 1.2.1: Imitation game (test di Turing)

Ci sono tre attori: un **umano**, un **computer** e un **interrogante**, collocato in una stanza diversa. L'interrogante deve classificare quale dei due interlocutori sia l'umano e quale la macchina, comunicando solo tramite testo.
Domanda: *è possibile che un computer inganni l'interrogante ed essere classificato come umano?*

Al test sono state mosse varie obiezioni; resta però il fatto che esso definisce implicitamente una *agenda di ricerca*.

Proposition 1.2.1 Capacità richieste dal test di Turing

Per superare il test, un calcolatore deve possedere:

(a) **Natural Language Processing** – comprendere e produrre linguaggio;

(b) **Knowledge Representation** – rappresentare ciò che sa;

(c) **Automatic Reasoning** – ragionare sulla conoscenza rappresentata;

(d) **Machine Learning** – adattarsi e generalizzare.

Oltre il test di Turing (interazione fisica): robotica, visione artificiale, parlato, movimento.

Example 1.2.1 (Chatbot e test di Turing)

Nel 2014 il chatbot **Eugene Goostman**, che imitava le risposte di un ragazzino di 13 anni, ha superato il test. Altri sistemi noti: **Cleverbot** (Machine Intelligence Prize 2010), **SIRI** (Apple), **Cortana** (Microsoft), **Alexa** (Amazon), **ChatGPT** (OpenAI).
Sono spesso costruiti indicizzando conversazioni precedenti: ciò che manca è la **coerenza** (da cui le *allucinazioni*), lo **stato del dialogo** e la **semantica**.

Note:

Prospettiva metodologica: l'approccio ingegneristico privilegia l'**emulazione** del comportamento intelligente in domini limitati, più che il raggiungimento di una intelligenza *reale*.

1.2.1 Oltre il test di Turing

Task che richiedono comprensione profonda del linguaggio, *common-sense*, capacità di ragionamento e analisi multimodale:

- **rompicapo matematici**;
- **risoluzione di problemi geometrici** (Allen Institute).

Example 1.2.2 (Puzzle dell'Allen Institute)

Usando i numeri interi da 2 a 7, riempire le caselle libere in modo che la somma dei tre numeri attorno al cerchio nero sia 11.

1.2.2 La previsione di Turing

Note:

"I believe that in about fifty years' time it will be possible to programme computers, with a storage capacity of about 10^9, to make them play the imitation game so well that an average interrogator will not have more than 70% chance of making the right identification after five minutes of questioning. ... I believe that at the end of the century the use of words and general educated opinion will have altered so much that one will be able to speak of machines thinking without expecting to be contradicted."

— A. Turing, *Computing machinery and intelligence*, Mind, vol. 59, n. 236, pp. 433–460, 1950.

1.3 Classificazione dei sistemi di IA

Definition 1.3.1: Weak AI vs Strong AI

- **Weak AI** – domanda di ricerca: *è possibile costruire sistemi che si comportano **come se** fossero intelligenti?*
- **Strong AI** – domanda di ricerca: *è possibile costruire sistemi che **sono** intelligenti?* Che abbiano menti coscienti, volontà e sentimenti?

Definition 1.3.2: General AI vs Narrow AI

- **General AI**: sistemi in grado di affrontare qualunque task generalizzato venga loro richiesto, molto come farebbe un umano.
- **Narrow AI**: sistemi in grado di gestire un *solo* task particolare. Mostrano un certo grado di intelligenza in un campo specifico, ma restano sistemi che eseguono compiti altamente specializzati per gli umani all'interno di quel campo ristretto.

1.4 Nascita e definizioni dell'IA

L'Intelligenza Artificiale nasce nel **1956** (Minsky, McCarthy, Shannon, Newell, Simon).

Note:

"The study is to proceed on the basis of the conjecture that every aspect of learning or any other feature of intelligence can in principle be so precisely described that a machine can be made to simulate it." [McCarthy 1955]

Alcune definizioni di IA:

- **Studio di come far fare ai computer le cose che gli umani fanno meglio.** È una definizione *transitoria*: si pensi agli scacchi, dove l'uomo non è più il migliore.
- **Studio di come costruire computer che superino il test di Turing** (ragionamento, comprensione del linguaggio naturale, apprendimento). Se il sistema è situato in un ambiente si aggiungono percezione, visione, movimento, robotica.
- Altre definizioni non si legano direttamente all'intelligenza umana, ma all'**abilità di interagire con un ambiente e di adattarvisi**.

1.5 Breve storia dell'IA

- **1943–1956 – La nascita dell'IA.** Reti neurali, programmi per gli scacchi, dimostratori di teoremi.
- **1952–1969 – Entusiasmo iniziale e grandi aspettative.** General Problem Solver, linguaggio LISP, reti neurali.
- **1966–1974 – Ritorno con i piedi per terra.** Alcuni programmi non erano adeguati (ELIZA, traduzioni sintattiche, reti neurali), altri erano troppo costosi computazionalmente (**esplosione combinatoria**).
- **1969–1979 – Sistemi basati su conoscenza.** Conoscenza profonda in un dominio limitato: **sistemi esperti**.
- **1980–1988 – L'IA entra nell'industria.** Sistemi esperti commercialmente di successo, progetto giapponese di *quinta generazione*, aziende che lavorano sull'IA, fondi di ricerca.
- **1988–2010 – Era del Web e di Internet.** Sistemi di supporto alle decisioni, agenti robotici, linguaggio naturale.
- **2010–oggi – Machine learning e deep learning.** Big data e potenza di calcolo massiva hanno reso funzionanti le reti profonde. Intelligenza e percezione aumentate, interesse industriale, industria 4.0.
- **2020–oggi – IA generativa e agenti conversazionali.**

1.6 I due approcci principali all'IA

Definition 1.6.1: Approccio top-down (simbolico)

- Rappresentazione **simbolica** della conoscenza;
- logiche, ontologie, sistemi a regole, architettura **dichiarativa**;
- modelli **comprensibili all'uomo**.

Definition 1.6.2: Approccio bottom-up (connessionista)

- **Reti neurali**. La conoscenza non è simbolica ed è "codificata" nelle connessioni tra i neuroni;
- i concetti sono **appresi da esempi**;
- i modelli **non sono comprensibili** all'uomo.

1.7 Ragionamento e logica

Definition 1.7.1: Ragionamento deduttivo

Sillogismo aristotelico, logica deduttiva. Da *Tutti gli uomini sono mortali* e *Socrate è un uomo* segue *Socrate è mortale*.
È **corretto**, ma non permette di "imparare" nuova conoscenza.

Definition 1.7.2: Ragionamento induttivo

Apprendimento, *deduzione inversa*. Dall'osservazione che diversi uccelli volano si conclude che *tutti gli uccelli volano* (e i pinguini?).
Produce **nuova conoscenza** a scapito della **correttezza**.

Definition 1.7.3: Ragionamento ipotetico o abduttivo

Duale del deduttivo. Dall'osservazione della *morte di Socrate* e sapendo che *tutti gli uomini sono mortali*, si ipotizza che *Socrate sia un uomo* (e se fosse un gatto?).
Risale alle **cause** attraverso l'osservazione degli **effetti**, a scapito della correttezza.

Definition 1.7.4: Ragionamento per analogia

Metaforico, *case-based*. Non richiede un modello né molti dati, ma usa il **principio di similarità**: Socrate e John "si assomigliano"; se Socrate ama la filosofia allora anche John ama la filosofia.
Algoritmi tipici: **K-Nearest-Neighbor** e **Support Vector Machine**.

Definition 1.7.5: Constraint reasoning e ottimizzazione

Ragionamento basato su **vincoli**, **probabilità** e **statistica** (teorema di Bayes).

1.8 Logica, dimostrazione automatica e Prolog

Lo schema concettuale che lega logica e dimostrazione automatica è il seguente: la **sintassi** manipola simboli *non interpretati*; la **semantica** fornisce un'*interpretazione*; i due mondi sono collegati dalle proprietà di **correttezza** (*soundness*) e **completezza**.

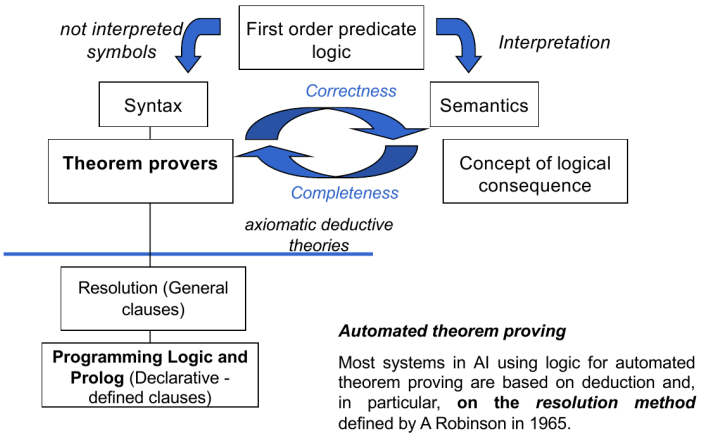


Figure 1.1: Dalla logica del primo ordine ai theorem prover e a Prolog.

Theorem 1.8.1 Risoluzione

La maggior parte dei sistemi di IA che usano la logica per la dimostrazione automatica di teoremi si basa sulla **deduzione** e, in particolare, sul **metodo di risoluzione** definito da A. Robinson nel **1965**.

Note:

La catena è: logica del primo ordine → teorie assiomatiche deduttive → risoluzione su **clausole generali** → programmazione logica e Prolog, che si restringe alle **clausole definite** (Horn) per ottenere efficienza.

1.8.1 Tre letture del concetto di algoritmo

Definition 1.8.1: Interpretazioni di "algoritmo"

1. Algoritmo = **strutture dati + istruzioni**;
2. Algoritmo = **logica (conoscenza) + controllo (motore inferenziale)**;
3. Algoritmo = **esempi (esperienza) + machine learning**.

Scendendo nella lista aumentano il **livello di generalità** e l'"**intelligenza**".

1.8.2 Linguaggi dichiarativi e Prolog

Definition 1.8.2: Algoritmo = Logica + Controllo

La conoscenza sul problema è **indipendente dal suo uso**:

- esprime il **COSA** e non il **COME**;
- alta **modularità** e **flessibilità**;
- è lo schema di progetto alla base della maggior parte dei **sistemi basati su conoscenza** (sistemi esperti).

LOGICA: la conoscenza sul problema determina *correttezza* ed *efficienza*.
CONTROLLO: la strategia di soluzione determina l'*efficienza*.

Note:

PROLOG = *PROgramming in LOGic*. È il più noto linguaggio di programmazione logica; nasce dalle idee di **R. Kowalski** e dalla prima realizzazione di **A. Colmerauer** (1973).

1.8.3 Struttura di un programma Prolog

Definition 1.8.3: Programma Prolog

Un programma Prolog è un insieme di **clausole di Horn** che rappresentano:

- **FATTI** sugli oggetti in esame e sulle relazioni;
- **REGOLE** su oggetti e relazioni (IF ... THEN);
- **GOAL** (clausole senza testa), sulla base della conoscenza definita.

Una regola ha la forma `Testa :- Corpo`.

Example 1.8.1 (Colleghi di lavoro)

Due individui sono colleghi se lavorano per la stessa azienda.

```
colleague(X,Y) :- works(X,Z), works(Y,Z), different(X,Y). % REGOLA

works(emp1,ibm).
works(emp2,ibm).
works(emp3,txt). % FATTI
works(emp4,olivetti).
works(emp5,txt).

:- colleague(X,Y). % GOAL
```

Example 1.8.2 (Somma di due interi (definizione dichiarativa))

```
sum(0,X,X). % FATTO
sum(s(X),Y,s(Z)) :- sum(X,Y,Z). % REGOLA
```

- `sum` è un simbolo **non interpretato**;
- i numeri interi sono definiti con la struttura **successore** `s(X)`;
- la definizione è **ricorsiva**.

Lo stesso programma ammette **molte query diverse**, cioè molti algoritmi:

```
:- sum(s(0),s(s(0)),Y). % somma
:- sum(s(0),Y,s(s(s(0)))). % sottrazione
:- sum(X,Y,s(s(s(0)))). % tutte le decomposizioni di 3
:- sum(X,Y,Z). % tutte le triple
:- sum(X,Y,s(s(s(0)))), sum(X,s(0),Y). % sistema di vincoli
```

Note:

Questo è il punto chiave della dichiaratività: **un solo programma, molti algoritmi**. Cambiando quali argomenti sono in input e quali in output si ottengono comportamenti diversi senza riscrivere il codice.

1.8.4 Deduzione e induzione in programmazione logica

Si usano tre insiemi: la **teoria** T (le regole), la **background knowledge** B (i fatti noti) e gli **esempi** E.

Definition 1.8.4: Deduzione

Dati T e B, si deriva E: $T \cup B \models E$.

T **(teoria)**	B **(background)**	E **(dedotto)**
`parent(X,Y):-mother(X,Y).`	`mother(mary,vinni).`	`parent(mary,vinni).`
`parent(X,Y):-father(X,Y).`	`mother(mary,andre).`	`parent(mary,andre).`
	`father(carrey,vinni).`	`parent(carrey,vinni).`
	`father(carrey,andre).`	`parent(carrey,andre).`

Definition 1.8.5: Induzione

Dati B e gli esempi **positivi** E^+ (e **negativi** E^-), si induce la teoria T:

E^+ / E^-	B **(background)**	T **(indotta)**
`parent(mary,vinni).`	`mother(mary,vinni).`	`parent(X,Y):-mother(X,Y).`
`parent(mary,andre).`	`mother(mary,andre).`	`parent(X,Y):-father(X,Y).`
`parent(carrey,vinni).`	`father(carrey,vinni).`	
`parent(carrey,andre).`	`father(carrey,andre).`	
`parent(mary,carrey).` $\in E^-$		

Le condizioni che T deve soddisfare sono:

$$\forall e^+ \in E^+ : \; B \cup T \models e^+ \qquad (T \text{ è } \textbf{completa})$$

$$\forall e^- \in E^- : \; B \cup T \not\models e^- \qquad (T \text{ è } \textbf{consistente})$$

Note:

L'induzione è dunque **deduzione inversa**: si cerca la teoria che, insieme alla background knowledge, *spiega* tutti gli esempi positivi e *nessuno* di quelli negativi. Questo è il cuore dell'**Inductive Logic Programming**.

1.8.5 Abduzione in programmazione logica

Definition 1.8.6: Abduzione

Dati la teoria T e le osservazioni E, si cercano le **ipotesi** B tali che $T \cup B \models E$.
Con T = {`parent(X,Y):-mother(X,Y).`, `parent(X,Y):-father(X,Y).`} e E = {`parent(mary,vinni),...`} si ottengono ipotesi **disgiuntive**:

```
mother(mary,vinni) v father(mary,vinni).
mother(mary,andre) v father(mary,andre).
mother(carrey,vinni) v father(carrey,vinni).
mother(carrey,andre) v father(carrey,andre).
```

Per **controllare le ipotesi** si usano spesso i **vincoli di integrità** (*integrity constraints*, IC).

Example 1.8.3 (Abduzione con vincoli di integrità)

Aggiungendo alla teoria i vincoli

```
mother(X,Y) -> female(X).
father(X,Y) -> male(X).
female(mary).
male(carrey).
```

le ipotesi disgiuntive collassano in ipotesi **determinate**:

```
mother(mary,vinni).
mother(mary,andre).
father(carrey,vinni).
father(carrey,andre).
```

Le condizioni sono ora: $T \cup B \models E$ e $T \cup B \models IC$.

1.8.6 Prolog: successi e limiti

- Grande idea rivoluzionaria: il linguaggio **non è legato all'architettura di Von Neumann**;
- contrapposizione **europea** vs **americana**;
- progetto giapponese di **quinta generazione** negli anni '80 (fallito);
- nel frattempo la tecnologia ha fatto passi da gigante...
- *perché Prolog non si è diffuso come si credeva negli anni '80?*
- molte applicazioni, idee, ricerca ed estensioni di Prolog nell'IA, ma non tutto.

Note:

Implementazioni notevoli:

- **SWI Prolog** – molto usato e ben integrato con il Semantic Web (http://www.swi-prolog.org);
- **tuProlog** – Prolog basato su Java per applicazioni Internet, sviluppato a Unibo (http://apice.unibo.it/xwiki/bin/view/Tuprolog/).

1.9 Machine learning

Imparare dall'esperienza, dai propri errori, dai maestri, per imitazione, è una caratteristica peculiare dell'intelligenza.
È un approccio **empirista**, che si oppone all'approccio **razionalista**.

Definition 1.9.1: Apprendimento (Michalski, 1986)

Learning is constructing or modifying representations of what is being experienced.

Definition 1.9.2: Apprendimento (Simon, 1984)

Learning denotes changes in the system that are adaptive in the sense that they enable the system to do the same task or tasks drawn from the same population more efficiently and more effectively the next time.

Note:

Il machine learning è uno dei temi caldi dell'IA: tutte le maggiori aziende ICT investono e assumono persone con background in ML (Google, Facebook, IBM, Baidu, Disney, ...). **Perché?** Perché hanno **molti dati** e vogliono **estrarne valore**: è l'era dei *big data*.

1.9.1 Le tre grandi famiglie

Definition 1.9.3: Supervised Learning

Parte da un insieme di esempi forniti da un "maestro" (il **training set**). Risolve problemi di **classificazione** e **regressione** (es. riconoscimento di pattern).

Definition 1.9.4: Unsupervised Learning

Procede per **osservazione e scoperta**. Dall'esterno non riceve alcun aiuto: è il sistema stesso a dover analizzare l'informazione disponibile, classificarla e strutturarla, scoprendo **pattern**. Task tipici: **clustering** e **data mining**.

Definition 1.9.5: Reinforcement Learning

Apprende il **comportamento ottimo** dalle esperienze passate. Osserva le scelte buone e cattive fatte (attraverso **ricompense** e **punizioni**) e modifica il comportamento di conseguenza. Impara principalmente **dagli errori**; trova molte applicazioni in robotica.

1.9.2 Che cosa si può fare con il machine learning?

- classificare le e-mail in arrivo come **spam** o non spam;
- predire il **prezzo delle azioni** Apple;
- formulare una **diagnosi** a partire dagli esami di un paziente;
- riconoscere **numeri di partita IVA** scritti a mano [LeCun et al., 1989];
- insegnare a un **robot** ad afferrare una tazza;
- progettare una **molecola** con proprietà specifiche;
- **tradurre** un testo;
- **convertire la voce** in testo;
- generare automaticamente la **didascalia** di una figura;
- **battere il campione del mondo** in un gioco.

1.9.3 Task: classificazione

Definition 1.9.6: Classificazione

La classificazione è il task di approssimare una funzione di mapping f da variabili di input X a variabili di output **discrete/categoriche** y.
Le variabili di output sono spesso chiamate **label**, **categorie** o **classi**. La funzione di mapping predice la classe o categoria di una data osservazione. È un task di apprendimento **supervisionato**.

Proposition 1.9.1 Varianti del problema di classificazione

- un problema di classificazione richiede che gli esempi siano classificati in una tra due o più classi;
- le variabili di input possono essere **reali** o **discrete**;
- con due classi si parla di **two-class** o **binary classification**; con più di due classi di **multi-class**;
- se a un esempio sono assegnate *più* classi si parla di **multi-label classification**;
- la **classification accuracy** è la percentuale di esempi correttamente classificati sul totale delle predizioni.

1.9.4 Task: regressione

Definition 1.9.7: Regressione

La regressione è il task di approssimare una funzione di mapping f da variabili di input X a una variabile di output **continua** y.
Una variabile di output continua è un valore reale (intero o floating point): tipicamente quantità, come importi e dimensioni. È un task **supervisionato**.

Proposition 1.9.2 Varianti del problema di regressione

- un problema di regressione richiede la predizione di una **quantità**;
- le variabili di input possono essere reali o discrete;
- con più variabili di input si parla di **multivariate regression**;
- se le variabili di input sono **ordinate nel tempo** si parla di **time series forecasting**;
- poiché il modello predice una quantità, la bontà del modello va riportata come **errore** sulle predizioni, ad esempio l'**RMSE**.

1.9.5 Task: clustering

Definition 1.9.8: Clustering

Il clustering è il problema di organizzare oggetti in **gruppi** i cui membri sono simili in qualche modo. Determina il **raggruppamento intrinseco** in un insieme di dati **non etichettati**: è il principale task di apprendimento **non supervisionato**.

Definition 1.9.9: Tipi di clustering

- **Hard clustering**: ogni elemento può essere assegnato a uno e un solo gruppo;
- **Soft** o **fuzzy clustering**: ogni elemento può appartenere a più gruppi.

L'algoritmo più usato per l'hard clustering è **k-means** (MacQueen, 1967).

1.10 Apprendimento simbolico e sub-simbolico

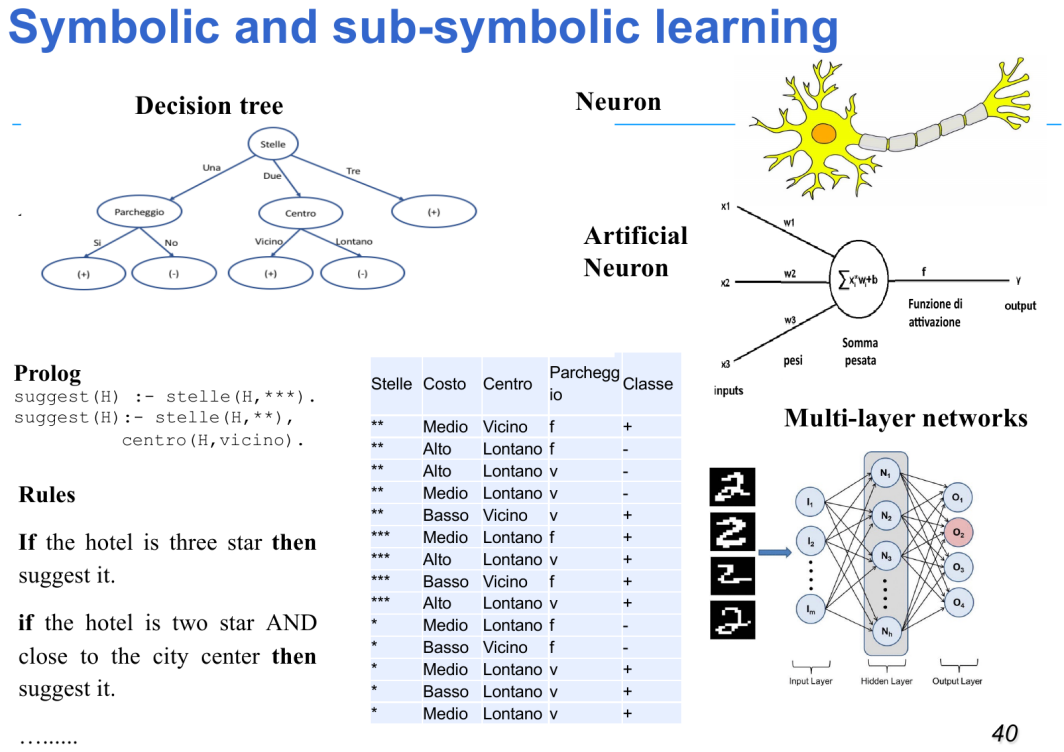


Figure 1.2: Due mondi a confronto: albero di decisione (e regole Prolog) contro neurone artificiale e reti multi-livello.

1.10.1 Apprendimento simbolico: alberi di decisione (ID3, C4.5)

Example 1.10.1 (Training set degli hotel)

Stelle	Costo	Centro	Parcheggio	Classe
**	Medio	Vicino	f	+
**	Alto	Lontano	f	−
**	Alto	Lontano	v	−
**	Medio	Lontano	v	−
**	Basso	Vicino	v	+
***	Medio	Lontano	f	+
***	Alto	Lontano	v	+
***	Basso	Vicino	f	+
***	Alto	Lontano	v	+
*	Medio	Lontano	f	−
*	Basso	Vicino	f	−
*	Medio	Lontano	v	+
*	Basso	Lontano	v	+
*	Medio	Lontano	v	+

L'albero di decisione appreso si traduce in **regole leggibili**:

- se l'hotel è a tre stelle allora suggeriscilo;
- se l'hotel è a due stelle **AND** è vicino al centro allora suggeriscilo;
- ...

che in Prolog diventano:

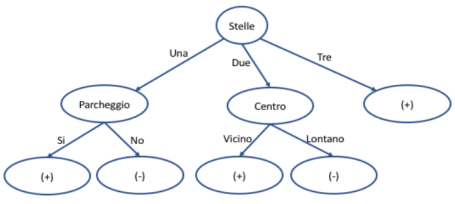
```
suggest(H) :- stelle(H,***).
suggest(H) :- stelle(H,**), centro(H,vicino).
```

Symbolic learning: decision trees (ID3, C4.5)

Possible training set of hotels:

Stelle	Costo	Centro	Parcheggio	Classe
**	Medio	Vicino	f	+
**	Alto	Lontano	f	-
**	Alto	Lontano	v	-
**	Medio	Lontano	v	-
**	Basso	Vicino	v	+
***	Medio	Lontano	f	+
***	Alto	Lontano	v	+
***	Basso	Vicino	f	+
***	Alto	Lontano	v	+
*	Medio	Lontano	f	-
*	Basso	Vicino	f	-
*	Medio	Lontano	v	+
*	Basso	Lontano	v	+
*	Medio	Lontano	v	+

Corresponding decision tree



Rules

If the hotel is three star **then** suggest it.

if the hotel is two star AND close to the city center **then** suggest it.

.....

41

Figure 1.3: Training set degli hotel e albero di decisione corrispondente.

Note:

Il punto di forza dell'approccio simbolico è qui evidente: il modello appreso è **ispezionabile** e traducibile in regole logiche. È esattamente ciò che si perde passando alle reti neurali.

1.10.2 Il neurone artificiale

L'idea è **simulare direttamente il funzionamento del cervello** in un calcolatore, e costruire una macchina intelligente a partire da neuroni artificiali. Il primo modello matematico di neurone artificiale, ispirato ai neuroni biologici, è stato proposto nel **1943** da **McCulloch e Pitts**.

Definition 1.10.1: Neurone artificiale

Un neurone riceve un insieme di **input**, ne fa una **somma pesata** e poi applica una **funzione di attivazione** per calcolare l'output.
L'output è controllato dalla funzione di attivazione: ogni neurone è **attivo** solo se il suo input supera una certa **soglia**.

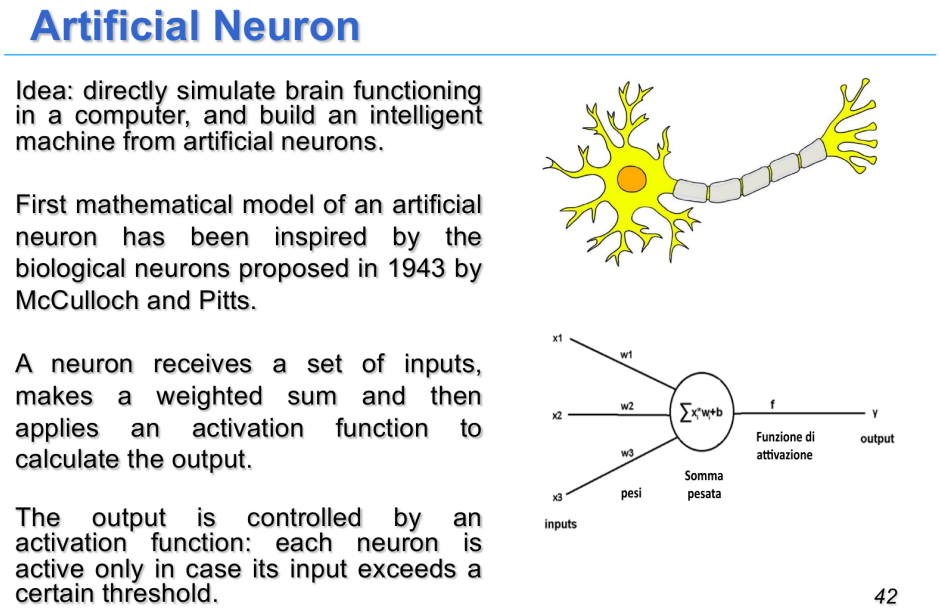


Figure 1.4: Neurone biologico e neurone artificiale: pesi, somma pesata, funzione di attivazione.

1.10.3 Il percettrone a due input

Definition 1.10.2: Percettrone

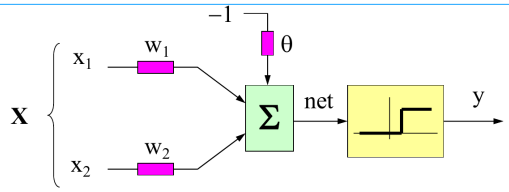
Con due input x_1, x_2, pesi w_1, w_2 e soglia θ:

$$y = H_S(w_1 x_1 + w_2 x_2 - \theta)$$

dove H_S è la funzione a gradino (Heaviside). I pattern appartenenti alla classe sono quelli tali che

$$w_1 x_1 + w_2 x_2 - \theta > 0.$$

Two input perceptron



$$y = HS\,(w_1\,x_1 + w_2\,x_2 - \theta)$$

The patterns belonging to the class will be those such that:

$$w_1\,x_1 + w_2\,x_2 - \theta > 0$$

43 43

Figure 1.5: Percettrone a due input.

Proposition 1.10.1 Separazione lineare dello spazio di input

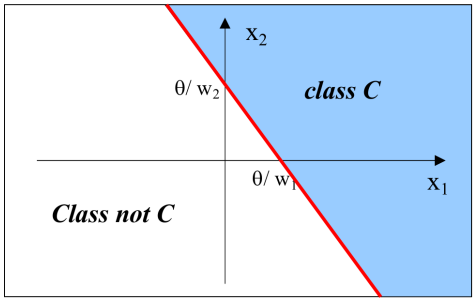
La condizione $w_1x_1 + w_2x_2 - \theta > 0$ si riscrive come

$$x_2 > -\frac{w_1}{w_2}\,x_1 + \frac{\theta}{w_2}$$

che è il semipiano sopra una **retta**: il percettrone realizza dunque una **separazione lineare** dello spazio di input. La retta interseca gli assi in θ/w_1 e θ/w_2.

Linear separation of the input space

$$x_2 > -(w_1/\,w_2)\,x_1 + \theta\,/\,w_2$$



44 44

Figure 1.6: La retta di separazione divide la classe C dalla classe non-C.

Example 1.10.2 (Apprendere la porta AND)

Con la regola di aggiornamento $\Delta w = \eta\, \delta\, x$ si arriva ai pesi

$$w_1 = 0.5, \qquad w_2 = 1, \qquad \theta = 1$$

che separano il punto $(1, 1)$ (unico output positivo dell'AND) da $(0, 0)$, $(0, 1)$ e $(1, 0)$.

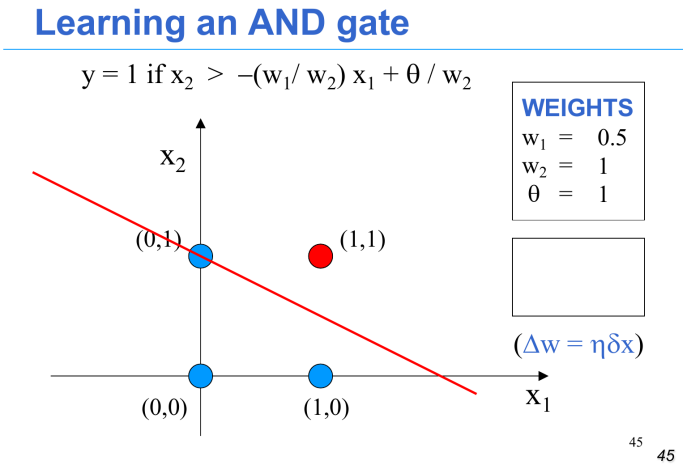


Figure 1.7: Apprendimento della porta AND: retta di separazione e pesi.

1.10.4 Reti neurali multi-livello

Per rappresentare concetti più complessi occorre costruire **reti di neuroni**: architetture **multi-livello** (reti *forward* o *feed-forward*) e nuovi algoritmi di apprendimento.

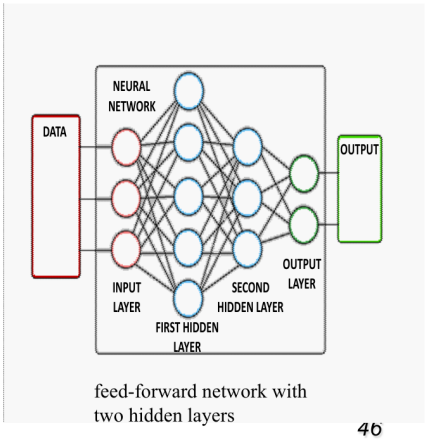
Note:

È un approccio **significativamente diverso** da quello simbolico: la conoscenza non è esplicita, ma è **insita nella struttura della rete e nei pesi delle connessioni**.

Neural Networks



- To represent more complex concepts networks of neurons should be built: multi-layer architectures (forward or feed-forward networks), and new learning algorithms.
- Significantly different approach from the symbolic one. The knowledge is not explicit but inherent in the network structure and connection weights.



feed-forward network with two hidden layers

46

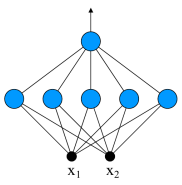
Figure 1.8: Rete feed-forward con due hidden layer.

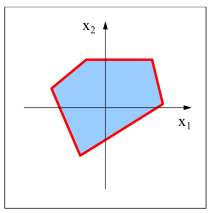
Proposition 1.10.2 Potere espressivo in funzione dei livelli

- **Reti a due livelli** (percettrone): separano solo regioni **linearmente separabili**;
- **Reti a tre livelli**: separano **regioni convesse**, con numero di lati $\leqslant$ numero di neuroni nascosti;
- **Reti a quattro livelli**: separano regioni di **qualsiasi forma**.

Three-layer networks

They are able to separate convex regions
number of edges ≤ number hidden neurons

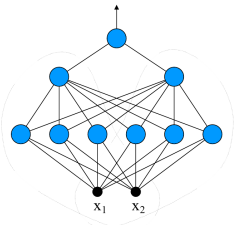


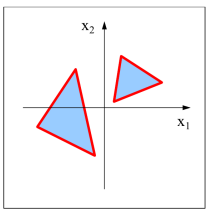


49 49

Four-layer networks

- They are able to separate regions of every shape





50 50

Figure 1.9: A sinistra: reti a tre livelli e regioni convesse. A destra: reti a quattro livelli e regioni di forma arbitraria.

Theorem 1.10.1 Teorema di approssimazione universale

Una rete **feed-forward** con **un solo hidden layer** e un **numero finito di neuroni** può approssimare **qualsiasi funzione continua** con l'accuratezza desiderata.

Note:

Teoria interessante, ma non dice **nulla** su come configurare la rete né su come applicare gli algoritmi di apprendimento per ottenere una buona approssimazione. Quali funzioni di output? Quanti neuroni? Quanti livelli?

Dagli anni '80 sono stati progettati modelli di neurone artificiale più complessi e dettagliati, con diverse funzioni **non lineari** (per esempio la **sigmoide**); sono state individuate varie **architetture** e sviluppati algoritmi di apprendimento più sofisticati.

1.11 Deep Neural Networks e Deep Learning

Definition 1.11.1: Deep Learning

Modelli e algoritmi che usano reti neurali con **molti neuroni** e **molti livelli**. Riescono ad apprendere funzioni complesse, cercando di individuare i dati più rilevanti.

Proposition 1.11.1 Caratteristiche delle DNN

- le DNN più usate hanno un numero molto elevato di livelli (anche qualche **centinaio**);
- il processo di **training è computazionalmente costoso**;
- gli algoritmi supervisionati attualmente ottengono buone prestazioni con circa **5 000 esempi per categoria** e **superano gli umani con 10 milioni di esempi**;
- **non** è una "simulazione" del cervello, che ha molti più neuroni e una struttura ben più complessa. Con un ulteriore sviluppo della potenza di calcolo si avrà un numero di neuroni confrontabile con quello umano nel **2050**.

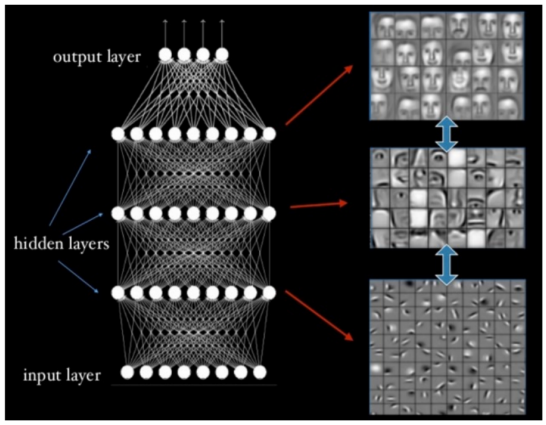
Da: I. Goodfellow, Y. Bengio, A. Courville, *Deep Learning*, MIT Press, 2016.

Note:

Nel Deep Learning i concetti sono appresi attraverso una **gerarchia di feature**, dalle più semplici alle più astratte e complesse. La rappresentazione è **implicitamente distribuita** nei diversi livelli.

Deep Neural Networks and Deep learning

In Deep Learning concepts are learnt through a hierarchy of features, from the simplest to more abstract and complex. Representation is implicitly distributed in different layers



https://leonardoaraujosantos.gitbooks.io/artificial-inteligence/content/deep_learning.html 53

Figure 1.10: Gerarchia di feature nel deep learning: dai bordi alle parti agli oggetti.

1.12 Foundation Models

Definition 1.12.1: Foundation model

Modelli su **larga scala** pre-addestrati su dati ampi e diversificati, poi **adattati** per alimentare innumerevoli applicazioni a valle. Sono il "base layer" dell'IA moderna.

Proposition 1.12.1 Tre caratteristiche dei foundation model

1. **Trained at Massive Scale** – apprendono da dataset ampi e diversificati che spaziano tra testo, codice, immagini e altro;
2. **Transformer Architecture** – la *self-attention* permette al modello di pesare le relazioni lungo sequenze di input molto lunghe;
3. **General-Purpose by Design** – un unico modello pre-addestrato si trasferisce a molti task tramite **fine-tuning** o **prompting**.

Traditional AI	Foundation Models
costruita per un task specifico	pre-addestrati una volta su dati ampi
addestrata da zero ogni volta	adattati via fine-tuning o prompting
richiede grandi dataset etichettati per ogni task	riusati su molti task e domini
trasferimento limitato a nuovi domini	capaci di uso *few-shot* o *zero-shot*

Note:

Esempi: GPT, Claude, Gemini, Llama, Stable Diffusion.

1.13 Agentic AI

Definition 1.13.1: Sistema agentico

I sistemi agentici **ragionano su un obiettivo**, **pianificano più passi**, **usano strumenti** e **si correggono** sulla base di ciò che osservano. Non si limitano a rispondere: *agiscono*.

Proposition 1.13.1 Il ciclo dell'agente

1. **Perceive / Observe** – raccoglie contesto dall'ambiente, dagli strumenti e dai passi precedenti;
2. **Plan** – scompone l'obiettivo in una sequenza di sotto-task e decisioni;
3. **Act** – chiama tool, API o altri sistemi per compiere azioni reali;
4. **Reflect** – valuta i risultati e adatta il piano, tornando al passo 1.

Applicazioni tipiche: *coding agent*, assistenti di ricerca, automazione di workflow, agenti di customer support.

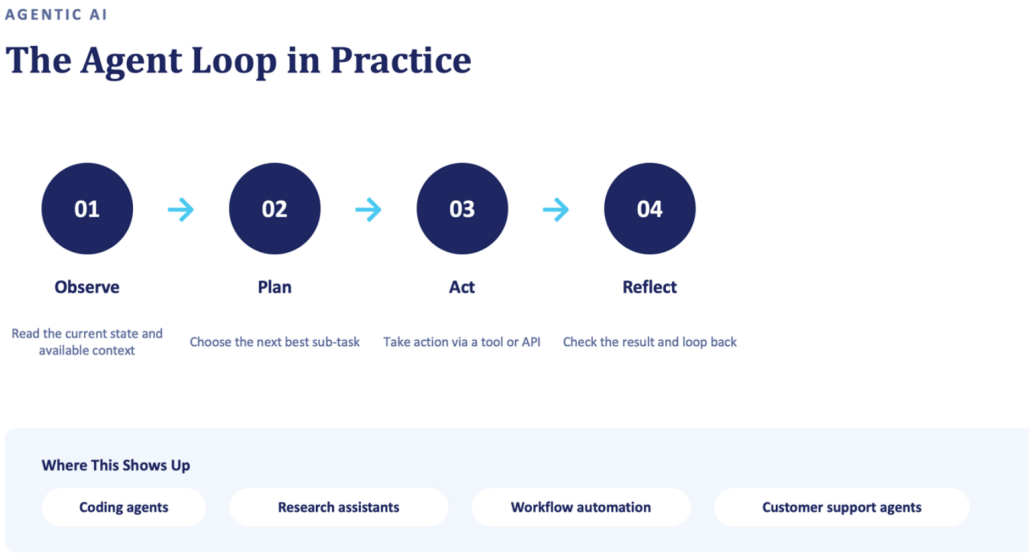


Figure 1.11: Il loop dell'agente: Observe → Plan → Act → Reflect.

Note:

Il ciclo agentico è, a ben vedere, una riedizione del **planning automatico** classico (stato iniziale, azioni, goal) in cui il motore di pianificazione è un foundation model invece di un planner simbolico.

1.14 Automated Planning

Definition 1.14.1: Planning automatico

Il planning automatico è un'importante attività di problem solving che consiste nel **sintetizzare una sequenza di azioni** eseguite da un agente, che porta da uno **stato iniziale** del mondo a un dato **stato obiettivo** (goal).
Dati:

- uno **stato iniziale**;
- un insieme di **azioni** eseguibili;
- uno **stato da raggiungere** (goal).

Trovare: un **piano**, cioè un insieme di azioni parzialmente o totalmente ordinato, necessario a raggiungere il goal dallo stato iniziale.

Definition 1.14.2: Planner

Un planner automatico è un agente intelligente che opera in un certo dominio, descritto da:

1. una **rappresentazione dello stato iniziale**;
2. una **rappresentazione del goal**;
3. una **descrizione formale delle azioni** eseguibili (dette anche **operatori**) in termini di **precondizioni** ed **effetti**.

Definisce dinamicamente il piano di azioni necessario a raggiungere il goal dallo stato iniziale.

1.14.1 Esempio: il mondo dei blocchi

Example 1.14.1 (Operatori del block world)

STACK(X,Y) – impila X su Y

- **IF**: `holding(X)` $\wedge$ `clear(Y)`
- **THEN**: `handempty` $\wedge$ `clear(X)` $\wedge$ `on(X,Y)`

UNSTACK(X,Y) – disimpila X da Y

- **IF**: `handempty` $\wedge$ `clear(X)` $\wedge$ `on(X,Y)`
- **THEN**: `holding(X)` $\wedge$ `clear(Y)`

PICKUP(X) – prendi X dal tavolo

- **IF**: `ontable(X)` $\wedge$ `clear(X)` $\wedge$ `handempty`
- **THEN**: `holding(X)`

PUTDOWN(X) – posa X sul tavolo

- **IF**: `holding(X)`
- **THEN**: `ontable(X)` $\wedge$ `clear(X)` $\wedge$ `handempty`

Note:

Si noti la struttura **precondizione → effetto** di ogni operatore: è esattamente la forma richiesta dalla definizione di planner. Le azioni sono duali a coppie (STACK/UNSTACK, PICKUP/PUTDOWN).

1.14.2 Tecniche di planning

- **Deductive planning / planning as search / linear planning** (trattati in *Foundations of AI*);
- **Nonlinear planning**, in particolare **Partial Order Planning** (POP);
- **Hierarchical planning**;
- **Conditional planning**;
- **Graph-based planning**;
- **Planning** per i **percorsi in robotica**;
- **Planning come comportamento emergente**: *swarm intelligence*;
- **Reinforcement learning** (per apprendere una *policy*).

1.15 Swarm intelligence e metafore naturali

La natura ha sviluppato varie forme di **intelligenza distribuita**:

- il nostro organismo, la selezione delle specie per adattarsi ai cambiamenti ambientali (**algoritmi genetici**);
- il **coordinamento tra animali** (formiche che costruiscono enormi nidi o spostano oggetti pesanti senza alcun coordinamento centrale) – **swarm intelligence**.

Note:

Chi governa tutto questo? Chi dà istruzioni, predice le dinamiche future, produce piani e mantiene l'equilibrio?
Nessuno: questi comportamenti intelligenti **emergono autonomamente**, senza coordinamento né supervisione centrale. Da qui lo sviluppo di sistemi intelligenti basati su **metafore naturali**, **robusti** e **adattivi**: *swarm intelligence* e *swarm robotics*.

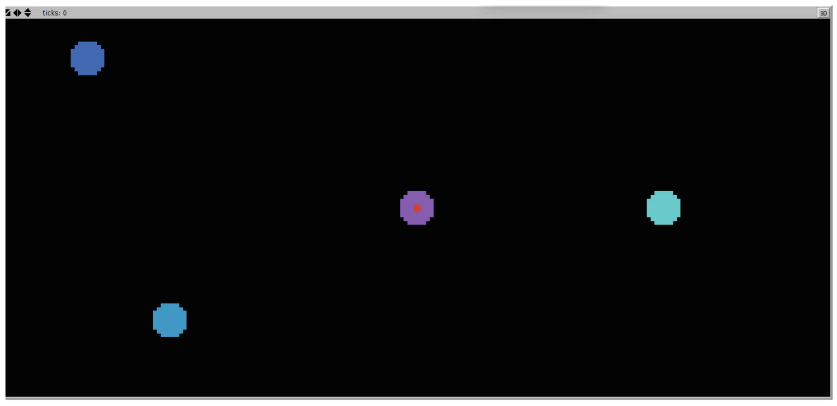


Figure 1.12: Formiche e ricerca del cibo: il cammino più breve emerge dal deposito di feromone, senza alcun controllo centrale.

1.15.1 Algoritmi genetici ed evolutivi

Definition 1.15.1: Algoritmo genetico

Gli algoritmi genetici (e più in generale gli algoritmi evolutivi) sono ispirati all'**evoluzione naturale** e sono stati sviluppati negli anni '70 da **John Holland**.
Si parte da una **popolazione iniziale casuale** di individui, che creano nuovi individui attraverso leggi specifiche (**cross-over** e **mutazione**).

Proposition 1.15.1 Operatori

- **Fitness**: garantisce che vengano mantenuti solo gli individui migliori;
- **Mutazione**: introduce nuovi elementi in modo casuale;
- **Cross-over**: combina buone soluzioni "genitore".

La fitness non è sempre facile da definire: in alcuni casi può essere **chiesta all'utente**.

Note:

Questi algoritmi sono utili quando è **difficile definire un modello** del problema.

1.16 Reinforcement learning

Definition 1.16.1: Reinforcement learning

Apprendimento di **policy**: come agire dato uno stato del sistema. Lo schema di controllo è:

$$X_k \longrightarrow \text{Learner} \xrightarrow{Y_k} \text{Controlled System} \xrightarrow{Z_k}$$

con un **Critic** che restituisce al learner il segnale di **rinforzo** R.
Rinforzo: punizione o ricompensa.

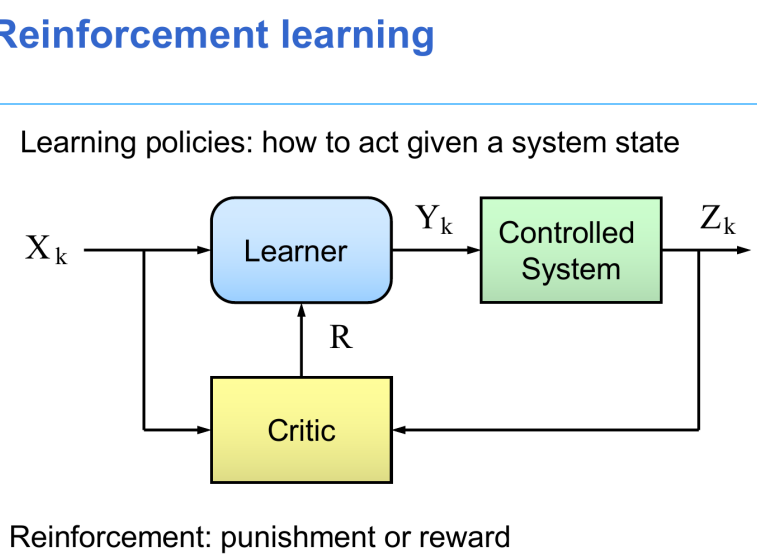


Figure 1.13: Schema del reinforcement learning: learner, sistema controllato e critic.

Note:

Applicazione emblematica: **battere il campione del mondo** in un gioco (AlphaGo, Google Research). Il RL chiude il cerchio del corso, perché è contemporaneamente una tecnica di **machine learning** e una tecnica di **planning** (apprendimento di una policy).

1.17 Riepilogo

Note:

Mappa concettuale della lezione:

- **Che cos'è l'IA**: intelligenza → test di Turing → weak/strong, general/narrow;
- **Due approcci**: simbolico (logica, regole, leggibile) vs connessionista (reti neurali, appreso da esempi, opaco);
- **Forme di ragionamento**: deduzione (corretta, non crea conoscenza), induzione (crea conoscenza, non corretta), abduzione (dalle cause agli effetti), analogia, vincoli;
- **Programmazione logica**: algoritmo = logica + controllo; Prolog, clausole di Horn, deduzione/induzione/abduzione nello stesso formalismo;

- **Machine learning**: supervisionato (classificazione, regressione), non supervisionato (clustering), con rinforzo;
- **Dal percettrone al deep learning**: separazione lineare → regioni convesse → regioni arbitrarie → gerarchie di feature → foundation model e agenti;
- **Planning**: stato iniziale, azioni con precondizioni ed effetti, goal; tecniche classiche e comportamenti emergenti.